Module: React Hooks, Routing, API Integration & Deploy

Phase: Frontend — React Advanced Lite

Shared topic (Python FS + Java FS).

Previous: React Fundamentals
Next (Python FS): Testing & Clean Code
Next (Java FS): Testing & Clean Code

Module Outcome:

- `useEffect` to fetch JSON, routes for more than one screen
- Talk to Django DRF or Spring Boot — and debug CORS instead of guessing
- Deploy the frontend (or document the blocked path) so the API is not localhost-only

Prerequisites

- React Fundamentals completed
- A local DRF Product API (or equivalent) you can call from the browser

Where used?
- CampusCart / **Swiggy** — product list `useEffect` + `/api/products`
- **Employee Management Portal** — employee directory fetch
- **Apollo Hospitals** — appointment calendar routes
- **HDFC Bank** — protected transfer route after login

CampusCart remains the capstone skin (college Swiggy).

Why useEffect?

`useState` holds data; **useEffect** runs code **after** React paints the screen — perfect for:

- Fetching products from Django when page loads
- Subscribing to timers (advanced)
- Syncing document title

```
useEffect(() => {
  // runs after first render
  fetchProducts();
}, []);  // empty array = run once on mount
```

If you put `fetchProducts()` directly in component body without useEffect, it would run every render and can cause infinite loops when setState inside.

Why React Router?

Without router, you toggle pages with `if (page === 'home')` — messy. **React Router** maps URLs to components:

- `/` → Home
- `/products` → Product list
- `/products/5` → Detail

Browser back button works. Users can bookmark `/products`.

Connecting Frontend to Backend

```
React (localhost:5173)  --fetch-->  Django DRF (localhost:8000/api/products/)
```

Different ports = different **origins**. Browsers block random cross-origin requests unless Django **allows** React origin — **CORS (Cross-Origin Resource Sharing)**.

Fix (Django): Install `django-cors-headers`, add middleware, set `CORS_ALLOWED_ORIGINS`.

Build & Deploy (overview)

- **Development:** `npm run dev` — hot reload
- **Production:** `npm run build` — optimized static files in `dist/`
- **Host React:** Netlify, Vercel, S3+CloudFront (static)
- **Host Django:** Render, Railway, AWS EC2, company server
- React `dist/` and Django API are separate deployments communicating over HTTPS

Concepts

useEffect patterns

Pattern	Dependency array	Use
On mount only	`[]`	Initial API fetch
On value change	`[productId]`	Refetch when id changes
Cleanup	return function	Cancel fetch, clear timer

Note:
- `useEffect` without deps — fetch every render; add `[]` or the correct dependency list.
- No loading state — blank screen confusion; show loading/error UI.
- `useEffect` async directly — warning patterns; use an inner async function instead.

Try these (2 programs):

Program 1 — Fetch once on mount

```
import { useEffect, useState } from "react";

export default function ProductList() {
  const [items, setItems] = useState([]);
  const [error, setError] = useState("");

  useEffect(() => {
    fetch("http://127.0.0.1:8000/api/products/")
```

```
      .then((r) => {
        if (!r.ok) throw new Error("Bad status");
        return r.json();
      })
      .then(setItems)
      .catch((e) => setError(e.message));
  }, []);

  if (error) return <p>Error: {error}</p>;
  return <ul>{items.map((p) => <li key={p.id}>{p.name}</li>)}</ul>;
}
```

Program 2 — Loading flag

```
import { useEffect, useState } from "react";

export default function LoaderDemo() {
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const t = setTimeout(() => setLoading(false), 500);
    return () => clearTimeout(t);
  }, []);
  return loading ? <p>Loading…</p> : <p>Ready</p>;
}
```

React Router v6 structure

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/products" element={<ProductListPage />} />
    <Route path="/products/:id" element={<ProductDetail />} />
  </Routes>
</BrowserRouter>
```

Note:
- Router outside `BrowserRouter` — hook/context errors; wrap the app once.

Try these (2 programs):

Program 1 — Basic routes

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Home() { return <h1>Home</h1>; }
function List() { return <h1>Products</h1>; }

export default function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/products">Products</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<List />} />
      </Routes>
    </BrowserRouter>
```

```
  );
}
```

Program 2 — Detail path

```
import { useParams } from "react-router-dom";

export default function ProductDetail() {
  const { id } = useParams();
  return <h1>Product #{id}</h1>;
}
// Route: <Route path="/products/:id" element={<ProductDetail />} />
```

API service layer

Centralize fetch logic in `src/api/products.js` — easier to change base URL and test.

Note:
- Wrong API URL in prod — 404 on deployed site; set `VITE_` env at build time.
- Forgot trailing slash for Django — 404 on `/products` vs `/products/`; match DRF URLs.

Try these (2 programs):

Program 1 — api.js helper

```
const BASE = import.meta.env.VITE_API_URL || "http://127.0.0.1:8000";

export async function listProducts() {
  const res = await fetch(`${BASE}/api/products/`);
  if (!res.ok) throw new Error("Failed to load products");
  return res.json();
}
```

Program 2 — Swiggy createProduct

```
export async function createProduct(payload) {
  const BASE = import.meta.env.VITE_API_URL || "http://127.0.0.1:8000";
  const res = await fetch(`${BASE}/api/products/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  if (!res.ok) throw new Error("Create failed");
  return res.json();
}
```

CORS headers (concept)

Django response includes `Access-Control-Allow-Origin: http://localhost:5173` so browser permits JavaScript to read response.

Note:
- Missing CORS config — browser CORS error while Postman still works; install and configure `django-cors-headers`.

Try these (2 programs):

Program 1 — Django CORS settings sketch

```
# settings.py
INSTALLED_APPS = [..., "corsheaders"]
MIDDLEWARE = ["corsheaders.middleware.CorsMiddleware", ...]
CORS_ALLOWED_ORIGINS = ["http://127.0.0.1:5173", "http://localhost:5173"]
```

Program 2 — Debug note

```
# If React shows CORS error

1. Is Django running?
2. Is corsheaders installed and middleware first-ish?
3. Is Vite origin listed in CORS_ALLOWED_ORIGINS?
Lab kit: labs/28_cors_fetch_debug/
```

Step-by-Step Annotated Code

Step 1 — Install dependencies

```
cd product-ui
npm install react-router-dom
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

`package.json`: add `"test": "vitest"` alongside existing Vite scripts.

Step 2 — Environment variable

`.env`:

```
VITE_API_URL=http://127.0.0.1:8000/api
```

`src/config.js`:

```
export const API_BASE = import.meta.env.VITE_API_URL || 'http://127.0.0.1:8000/api';
```

Step 3 — API module

`src/api/products.js`:

```
import { API_BASE } from '../config';

export async function getProducts() {
  const res = await fetch(`${API_BASE}/products/`);
  if (!res.ok) throw new Error(`Failed to load products (${res.status})`);
  return res.json();
}

export async function createProduct(payload) {
  const res = await fetch(`${API_BASE}/products/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
```

```
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(err.detail || 'Create failed');
  }
  return res.json();
}

export async function deleteProduct(id) {
  const res = await fetch(`${API_BASE}/products/${id}/`, { method: 'DELETE' });
  if (!res.ok && res.status !== 204) throw new Error('Delete failed');
}
```

Step 4 — Product list page with useEffect

`ProductListPage.jsx`: on mount, `useEffect` calls `getProducts()`, sets loading/error states, renders list with `Link` and delete button. Use cleanup flag to avoid setState after unmount.

Step 5 — Router in main.jsx / App.jsx

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Home from './pages/Home';
import ProductListPage from './pages/ProductListPage';
import ProductDetail from './pages/ProductDetail';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/products">Products</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<ProductListPage />} />
        <Route path="/products/:id" element={<ProductDetail />} />
      </Routes>
    </BrowserRouter>
  );
}
```

`ProductDetail.jsx` uses `useParams()` and `useEffect` with `[id]` dependency to fetch one product.

Step 6 — Django CORS setup

```
pip install django-cors-headers
```

`settings.py`:

```
INSTALLED_APPS = [
    # ...
    'corsheaders',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',  # high in list
    # ...
]
```

```
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5173',
    'http://127.0.0.1:5173',
]
```

Restart Django server after change.

Step 7 — Vitest lite test

One test: render component showing "Loading products..."; assert text with `screen.getByText`. Run: `npm test`

> **Note:**
> - Test fetch without mock — flaky tests; mock the API module in tests.

Step 8 — Build overview

```
npm run build    # creates dist/
npm run preview  # local preview of production build
```

Deploy `dist/` to Netlify; set production `VITE_API_URL` to hosted Django URL in Netlify env settings.

Practice Exercises

Basic

1. Add Home page with link to products and course name.
   Model answer:

```
import { Link } from "react-router-dom";

export default function Home() {
  return (
    <main>
      <h1>Python Full Stack</h1>
      <Link to="/products">View products</Link>
    </main>
  );
}
```

2. ProductDetail shows name, price, stock from API by id.
   Model answer:

```
const { id } = useParams();
const [product, setProduct] = useState(null);
useEffect(() => {
  fetch(`/api/products/${id}/`)
    .then((r) => r.json())
    .then(setProduct);
}, [id]);
if (!product) return <p>Loading…</p>;
return (
  <div>
    <h1>{product.name}</h1>
    <p>₹{product.price}</p>
    <p>Stock: {product.stock}</p>
```

```
  </div>
);
```

3. Document CORS error screenshot + fix in README.
   Model answer:
   Screenshot the browser console CORS error. Fix by allowing the React origin on the API (Django `CORS_ALLOWED_ORIGINS` or Flask-CORS). README: cause → fix setting → verify `GET` succeeds.

Intermediate

1. Create form page `POST` new product then navigate to list.
   Model answer:

```
async function onSubmit(e) {
  e.preventDefault();
  await fetch("/api/products/", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, price, stock }),
  });
  navigate("/products");
}
```

2. Refetch list when navigating back using location key or manual refresh button.
   Model answer:
   Depend `useEffect` on `location.key` from `useLocation()`, or add a Refresh button that re-calls the fetch function. Either approach reloads products after create/delete.
3. Add Vitest test for error message component.
   Model answer:

```
import { render, screen } from "@testing-library/react";
import { ErrorMessage } from "./ErrorMessage";

test("shows message", () => {
  render(<ErrorMessage text="Network failed" />);
  expect(screen.getByText("Network failed")).toBeInTheDocument();
});
```

Interview-style

1. What is CORS and who enforces it?
   Model answer:
   CORS is a browser security policy controlling cross-origin HTTP access. The browser enforces it using `Access-Control-*` headers from the server; tools like curl are not restricted the same way.
2. useEffect dependency array purpose?
   Model answer:
   It lists values that, when changed, re-run the effect. `[]` runs once after mount; omitting the array re-runs every render — usually a bug.
3. Difference between `npm run dev` and `npm run build`?
   Model answer:
   `dev` starts a local dev server with hot reload. `build` produces optimized static files in `dist/` / `build/` for production hosting.

Mini-Project (Assignment): Full-Stack Product Browser

Lab kits: 05_react_product_list · 28_cors_fetch_debug · 04_drf_products

Stack: React (Vite) + Django DRF Product API

What to build

- Routes: Home, List, Detail, Add Product
- Loading/error states
- Delete from list
- CORS configured
- One Vitest test
- README: how to run both servers

Stretch

- Show price in ₹ formatting on cards.
- Add a simple search box that filters client-side.

Done when

- [ ] React lists products from Django API
- [ ] CORS error is resolved (lab kit if needed)
- [ ] Add + delete work against the API
- [ ] README documents both `runserver` and `npm run dev`

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why does Postman work when browser fetch fails (same API)?
2. Should API URL be hardcoded in 10 components? Better pattern?
3. how is React Router like tabs in Excel but with URLs?
4. What changes when app goes from HTTP localhost to HTTPS production?
5. Is testing frontend worth it early in your career? What to test first?

Q1. What is useEffect?
A: React hook for side effects after render — data fetching, subscriptions, DOM sync — with optional dependency control.

Q2. Explain CORS simply.
A: Browser policy blocking JS from reading cross-origin responses unless server sends allow headers.

Q3. How React talks to Django?
A: HTTP requests (fetch/axios) to DRF JSON endpoints; Django handles data; React renders UI.

Q4. What happens in npm run build?
A: Vite bundles/minifies JS/CSS into static files for production hosting.

Q5. Vitest vs Jest?
A: Both test runners; Vitest integrates natively with Vite projects; similar API for unit/component tests.